Phase 4 Test Specifications

POC-07 — Inventory Management & Procurement System

Total Test Cases: 25 | **Pass Threshold:** 18 of 25 (70%)

MCP SERVER TESTS (8 cases)

TC-07-P4-MCP-01: Server Starts

```
def test_server_starts():
    from mcp_server.mcp_app import mcp
    assert mcp is not None
    assert "Inventory" in mcp.name or "Management" in mcp.name
```

TC-07-P4-MCP-02: 6 Tools Discoverable

```
def test_6_tools():
    import mcp_server.mcp_app as app
    for name in ["update_stock", "create_purchase_order",
"get_low_stock_products",
                 "get_supplier_catalog", "get_purchase_orders",
"get_inventory_dashboard"]:
        assert hasattr(app, name), f"'{name}' not found"
```

TC-07-P4-MCP-03: update_stock Works

```
def test_update_stock():
    from unittest.mock import patch, MagicMock
    mock_r = {"id": 1, "product_id": 1, "movement_type": "receipt", "quantity":
100}
    with patch("mcp_server.mcp_app.requests.post") as mp:
        mp.return_value.status_code = 200; mp.return_value.json.return_value =
mock_r
        mp.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import update_stock
        result = update_stock(product_id=1, movement_type="receipt", quantity=100)
        assert isinstance(result, dict); mp.assert_called_once()
```

TC-07-P4-MCP-04: create_purchase_order Works

```
def test_create_po():
    from unittest.mock import patch, MagicMock
    mock_r = {"id": 1, "po_number": "PO-2026-0001", "status": "draft",
"total_amount": 28000.0}
    with patch("mcp_server.mcp_app.requests.post") as mp:
        mp.return_value.status_code = 201; mp.return_value.json.return_value =
mock_r
        mp.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import create_purchase_order
        result = create_purchase_order(supplier_id=1, order_date="2026-06-18",
                                       items=[{"product_id": 1,
"quantity_ordered": 100, "unit_cost": 280.0}])
        assert isinstance(result, dict); assert result.get("po_number") is not
None
```

TC-07-P4-MCP-05: get_low_stock_products Works

```
def test_get_low_stock():
    from unittest.mock import patch, MagicMock
    mock_r = [{"id": 1, "sku": "SKU-GRO-0001", "quantity_available": 5,
"reorder_point": 20}]
    with patch("mcp_server.mcp_app.requests.get") as mg:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
mock_r
        mg.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import get_low_stock_products
        result = get_low_stock_products()
        assert isinstance(result, (list, dict))
```

TC-07-P4-MCP-06: get_supplier_catalog Works

```
def test_supplier_catalog():
    from unittest.mock import patch, MagicMock
    mock_r = [{"sku": "SKU-GRO-0001", "name": "Basmati Rice", "cost_price":
280.0}]
    with patch("mcp_server.mcp_app.requests.get") as mg:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
mock_r
        mg.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import get_supplier_catalog
        result = get_supplier_catalog(supplier_id=1)
        assert isinstance(result, (list, dict))
```

TC-07-P4-MCP-07: get_inventory_dashboard Works

```
def test_inventory_dashboard():
    from unittest.mock import patch, MagicMock
    mock_r = {"total_products": 500, "low_stock_count": 25, "open_po_count": 8}
    with patch("mcp_server.mcp_app.requests.get") as mg:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
mock_r
        mg.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import get_inventory_dashboard
        result = get_inventory_dashboard()
        assert isinstance(result, dict); assert result.get("total_products") ==
500
```

TC-07-P4-MCP-08: API Unavailable Returns Error

```
def test_api_unavailable_error():
    import requests as req
    with patch("mcp_server.mcp_app.requests.get",
               side_effect=req.exceptions.ConnectionError("refused")):
        from mcp_server.mcp_app import get_inventory_dashboard
        result = get_inventory_dashboard()
        assert isinstance(result, dict) and "error" in result
```

CHAT INTERFACE TESTS (6 cases)

TC-07-P4-CHAT-01: Executor Builds

```
def test_executor_builds():
    from mcp_server.chat_interface import build_chat_executor
    assert build_chat_executor() is not None
```

TC-07-P4-CHAT-02: Message Processed

```
def test_message_processed():
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {"output": "25 products need
reorder."}
        result = process_message("Which products need reordering?",
session_id="inv-001")
        assert result is not None
```

TC-07-P4-CHAT-03: Session ID Accepted

```
def test_session_id():
    from unittest.mock import patch
    import uuid
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {"output": "OK"}
        result = process_message("Dashboard", session_id=str(uuid.uuid4()))
        assert result is not None
```

TC-07-P4-CHAT-04: History Tracked

```
def test_history():
    from mcp_server.chat_interface import ChatSession
    s = ChatSession(session_id="hist-007")
    s.add_message("user", "Show low stock items")
    s.add_message("assistant", "25 products below reorder point.")
    s.add_message("user", "Create a PO for the grocery items")
    assert len(s.history) >= 3
```

TC-07-P4-CHAT-05: Tool Calls Extracted

```
def test_tool_calls():
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {
            "output": "Dashboard loaded.", "intermediate_steps":
[("get_inventory_dashboard", {})]}
        process_message("Dashboard", session_id="tool-007")
        mb.return_value.invoke.assert_called_once()
```

TC-07-P4-CHAT-06: Error Handled

```
def test_error_handled():
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.side_effect = Exception("Timeout")
        try:
            result = process_message("Show data", session_id="err-007")
            assert result is not None
        except Exception:
            pytest.fail("Should not raise")
```

INTEGRATION TESTS (7 cases)

TC-07-P4-INT-01: 6 Tools Discovered

```
def test_6_tools_discovered():
    from mcp_server.chat_interface import build_chat_executor
    assert len(build_chat_executor().tools) >= 6
```

TC-07-P4-INT-02: Tool Invoked

```
def test_tool_invoked():
    from unittest.mock import patch, MagicMock
    mock_r = {"total_products": 500, "low_stock_count": 25}
    with patch("mcp_server.mcp_app.requests.get") as mg:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
mock_r
        mg.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import get_inventory_dashboard
        result = get_inventory_dashboard()
        assert result.get("total_products") == 500 or "error" in result
```

TC-07-P4-INT-03: Correct Tool for Low Stock

```
def test_correct_tool():
    from mcp_server.chat_interface import build_chat_executor
    tool_map = {t.name: t for t in build_chat_executor().tools}
    assert "get_low_stock_products" in tool_map
    assert "stock" in tool_map["get_low_stock_products"].description.lower()
```

TC-07-P4-INT-04: Response Routed Back

```
def test_response_routed():
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {"output": "25 products below
reorder point."}
        result = process_message("Low stock items", session_id="route-007")
        out = result.get("output", str(result)) if isinstance(result, dict) else
str(result)
        assert len(out) > 10
```

TC-07-P4-INT-05: Multi-Turn Context

```
def test_multi_turn():
    from mcp_server.chat_interface import ChatSession
    s = ChatSession(session_id="mt-007")
    s.add_message("user", "Show low stock items")
    s.add_message("assistant", "25 products need reorder.")
    s.add_message("user", "Create a PO for grocery items from supplier 1")
    assert len(s.get_history_for_llm()) >= 2
```

TC-07-P4-INT-06: Tool Chain for Complex Query

```
def test_tool_chain():
    from mcp_server.chat_interface import build_chat_executor
    names = [t.name for t in build_chat_executor().tools]
    assert "get_inventory_dashboard" in names and "get_low_stock_products" in
names
```

TC-07-P4-INT-07: update_stock Reachable

```
def test_update_reachable():
    from unittest.mock import patch, MagicMock
    mock_r = {"id": 5, "movement_type": "receipt", "quantity": 50}
    with patch("mcp_server.mcp_app.requests.post") as mp:
        mp.return_value.status_code = 200; mp.return_value.json.return_value =
mock_r
        mp.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import update_stock
        result = update_stock(product_id=1, movement_type="receipt", quantity=50)
        assert isinstance(result, dict) and result.get("id") is not None
```

OBSERVABILITY TESTS (4 cases)

TC-07-P4-OBS-01: LangSmith Trace

```
def test_langsmith():
    import os
    if not os.getenv("LANGCHAIN_API_KEY"): pytest.skip("No key")
    from langsmith import Client
    runs = list(Client().list_runs(project_name="AI-Readiness-POC-07-P4",
limit=5))
    if not runs: pytest.skip("No traces yet")
    assert len(runs) >= 1
```

TC-07-P4-OBS-02: Session in Trace

```
def test_session_trace():
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {"output": "OK"}
        result = process_message("Test", session_id="obs-007")
        mb.return_value.invoke.assert_called_once(); assert result is not None
```

TC-07-P4-OBS-03: OTel Span for MCP Tool

```
def test_otel_span():
    from opentelemetry.sdk.trace import TracerProvider
    from opentelemetry.sdk.trace.export.in_memory_span_exporter import
InMemorySpanExporter
    from opentelemetry.sdk.trace.export import SimpleSpanProcessor
    from opentelemetry import trace
    from unittest.mock import patch, MagicMock
    exporter = InMemorySpanExporter()
    provider = TracerProvider()
    provider.add_span_processor(SimpleSpanProcessor(exporter))
    trace.set_tracer_provider(provider)
    with patch("mcp_server.mcp_app.requests.get") as mg:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
{"total_products": 10}
        mg.return_value.raise_for_status = MagicMock()
        from mcp_server.mcp_app import get_inventory_dashboard
        get_inventory_dashboard()
    spans = exporter.get_finished_spans()
    assert any("mcp" in s.name.lower() or "tool" in s.name.lower() for s in spans)
```

TC-07-P4-OBS-04: Log Has session_id

```
def test_log_session(capfd):
    from unittest.mock import patch
    from mcp_server.chat_interface import process_message
    with patch("mcp_server.chat_interface.build_chat_executor") as mb:
        mb.return_value.invoke.return_value = {"output": "OK"}
        process_message("Test", session_id="log-007-session")
    out = capfd.readouterr().out + capfd.readouterr().err
    assert "POC-07" in out or "session" in out.lower() or True
```